PRODUCT REQUIREMENTS DOCUMENT (PRD)

IPAM CONNECT

STUDENT NETWORKING & COMMUNICATION PLATFORM

Document Version: 1.2 (Vanilla JS Edition) | **Target Delivery:** Hackathon MVP | **Tech Stack:** HTML5, CSS3, ES6+ JavaScript, Firebase Suite, Vercel

1. Executive Summary & Problem Space

1.1 Problem Statement

University students face fragmented communication across disjointed social media platforms (WhatsApp, Telegram). This fragmentation isolates academic resources, obfuscates official communications, and restricts inter-departmental networking, resulting in lost collaboration opportunities and decentralized material sharing.

1.2 Proposed Solution

IPAM CONNECT is a centralized, academic-focused communication and networking ecosystem built without heavy framework layers to maximize performance and agility. By combining structured directory profiles, role-based real-time communication channels, and an instant-access academic repository, the platform streamlines student interaction and democratizes resource sharing via native browser technologies.

2. User Roles & Permission Matrix

The platform enforces a strict Role-Based Access Control (RBAC) model to ensure architectural security and distributed moderation capabilities across client-side views.

Capability / Feature Space	Student	Class Representative	Admin / Moderator
Register with Personal Email	YES	YES	YES
Search Student Directory & Edit Profile	YES	YES	YES
Send Direct Messages (with attachments)	YES	YES	YES
Upload Academic Resources (Instant Live)	YES	YES	YES
Post Class-Specific Announcements	NO	YES	YES
Delete Group Messages / Resource Assets	NO	YES (Own Class Only)	YES (System-Wide)
Manage Interest-Based Communities	NO	NO	YES

3. Epics & Functional Requirements

Epic 1: Identity & Profile Management (User Management)

- **FR-1.1: Registration Engine:** Users register securely via personal email accounts directly utilizing the native Firebase Authentication Web SDK (`firebase/auth`).
- **FR-1.2: Token-Based Role Elevation:** Sign-ups map default profiles to a "Student" tier. A conditional registration element accepts an Admin-Generated Single-Use Token. Upon execution, vanilla script handles token ingestion, cross-references Firestore, provisions "Class Representative" privileges, and invalidates the token permanently.
- **FR-1.3: Data-Sanitized Academic Profiling:** The onboarding UI mandates selection fields using strict, pre-defined native HTML `<select>` dropdown elements for **Departments**/**Courses** and **Year**/**Levels**, altogether eliminating text-entry validation errors and ensuring database consistency.
- **FR-1.4: Searchable Directory:** A fast DOM-rendered lookup grid matching structured user documents, enabling instant peer discovery filterable strictly by explicit department and level metrics.

Epic 2: Structured Real-Time Communication & Communities

- **FR-2.1: Automated Group Creation:** Authenticated Class Representatives can initialize authorized communication spaces tied directly to their assigned course arrays.
- **FR-2.2: Interest-Based Group Gatekeeping:** Custom affinity channels require a system dashboard toggle approval by a central Administrator prior to initialization.
- **FR-2.3: Reactive Direct Messaging (DM):** Instantaneous peer-to-peer text synchronization leveraging Cloud Firestore active snapshot listeners (`onSnapshot`) without needing persistent custom socket layers.
- **FR-2.4: Managed Chat Assets:** Binary payload transmission (Images, Documents, ZIP bundles) up to **50MB** routed straight to Firebase Storage buckets, updating reactive chat feeds with the resultant secure download links.
- **FR-2.5: Tiered Broadcast Feed:** Global announcements are restricted to central Admins, whereas class-level feeds permit targeted pin boards curated by Class Reps to reach corresponding students.

Epic 3: Centralized Academic Repository (Resource Sharing)

- **FR-3.1: High-Velocity Uploads:** Immediate file uploads to maximize crowd-sourced academic velocity, updating the directory layout dynamically in real-time.
- **FR-3.2: Egress & Optimization Guardrails:** Accepts materials up to **50MB**. The file input layout features an intentional warning callout instructing students to explicitly purge unoptimized items (such as local development `node_modules` configurations) before archiving to safe-keep cloud egress limits.
- **FR-3.3: Metadata Taxonomy:** Every uploaded element is programmatically structured with categorical select options, course codes, file sizes, and creation timestamps.
- **FR-3.4: Discovery UI:** High-speed string search filtering client-side metadata variables to ensure zero-latency resource location.

Epic 4: Distributed Emergency Moderation

- **FR-4.1: Live Chat Purging:** Class Representatives have explicit UI parameters to soft-delete problematic payloads inside their groups, updates flags to `isDeleted: true` to dynamically rewrite active chat strings.
- **FR-4.2: Resource Take-Down Engine:** Immediate client-side purging capabilities for Class Reps over materials published within their active departments to maintain a compliant, productive study landscape.

4. Technical Architecture & Data Strategy

4.1 System Topology

A decoupled, entirely serverless client-centric framework minimizing computational layout pipelines and emphasizing lightning-fast page paints directly via raw DOM APIs.

4.2 Data Schema Outline (Firestore Structures)

`tokens` Collection (Single-Use Verification Layer)

```
{
  "tokenId": "X9B2R1",
  "isConsumed": true,
  "assignedDepartment": "BSc Information Technology",
  "assignedLevel": "Year 2",
  "consumedBy": "STRING_AUTH_ID",
  "invalidatedAt": "TIMESTAMP"
}
```

`users` Collection

```
{
  "uid": "STRING_AUTH_ID",
  "fullName": "John Doe",
  "email": "johndoe@gmail.com",
  "role": "student | class_rep | admin",
  "studentId": "IPAM-2026-XXXX",
  "department": "BSc Information Technology",
  "level": "Year 2",
  "createdAt": "TIMESTAMP"
}
```

`messages` Collection

```
{
  "messageId": "STRING_MSG_ID",
  "senderId": "STRING_AUTH_ID",
  "text": "This message was deleted by a moderator",
  "attachmentUrl": "https://firebasestorage.googleapis.com/.../project.zip",
  "attachmentType": "application/zip",
  "isDeleted": true,
  "timestamp": "TIMESTAMP"
}
```

5. Hackathon MVP Release Scope Matrix

> **Scope Management Strategy:** To protect the continuous operational integrity of the system during the accelerated hackathon schedule, product milestones rigorously prioritize architectural logic stability over decorative components.

🟩 IN SCOPE (Must-Haves for Launch)

- Native Firebase Auth handling linked to single-use validation paths.
- Strict HTML selection input frameworks guaranteeing precise index data tagging.
- Direct messaging and group discussions leveraging native web database streaming.
- 50MB limit asset uploading equipped with optimization warning indicators.
- Class broadcast matrices and local deletion access configurations for Class Reps.

🟥 OUT OF SCOPE (Post-Hackathon Roadmap)

- Automated background lookups linking directly into institutional university registrar databases.
- Custom desktop background workers or custom push worker service configurations.
- Synchronous web audio/video channel integrations or visual stream canvas components.